

```

"""
실습 #1 – Simple Policy Gradient (CartPole)
PDF: 10-Policy Gradient Method – slides 6~10

 $\nabla_{\theta} J(\theta) \approx \sum_t G(\tau) \nabla_{\theta} \log \pi_{\theta}(A_t|S_t)$ 
loss = -  $\sum_t G(\tau) \log \pi_{\theta}(A_t|S_t)$ 

실행:
python Simple_pg2.py
python Simple_pg2.py --plot
"""
from __future__ import annotations

import argparse

import matplotlib.pyplot as plt
import numpy as np

try:
    import gym
except ModuleNotFoundError:
    import gymnasium as gym

from dezero import Model
from dezero import optimizers
import dezero.functions as F
import dezero.layers as L

class Policy(Model):
    """Policy Network ( $\pi_{\theta}$ ): 2-layer NN, CartPole classification."""

    def __init__(self, action_size: int = 2):
        super().__init__()
        self.l1 = L.Linear(128)
        self.l2 = L.Linear(action_size)

    def forward(self, x):
        x = F.relu(self.l1(x))
        x = F.softmax(self.l2(x))
        return x

class Agent:
    def __init__(self, action_size: int = 2, gamma: float = 0.98, lr: float = 0.0002):
        self.gamma = gamma
        self.action_size = action_size

```

```

self.memory: list[tuple[float, object]] = []

self.pi = Policy(action_size)
self.optimizer = optimizers.Adam(lr)
self.optimizer.setup(self.pi)

def get_action(self, state: np.ndarray) -> tuple[int, object]:
    state = state[np.newaxis, :]
    probs = self.pi(state)
    probs = probs[0]
    action = int(np.random.choice(self.action_size, p=probs.data))
    return action, probs[action]

def add(self, reward: float, prob) -> None:
    self.memory.append((reward, prob))

def update(self) -> None:
    self.pi.cleargrads()

    g_tau = 0.0
    for reward, _ in reversed(self.memory):
        g_tau = float(reward + self.gamma * g_tau)

    loss = 0
    for _, prob in self.memory:
        loss += -F.log(prob) * g_tau

    loss.backward()
    self.optimizer.update()
    self.memory.clear()

def moving_average(xs: list[float], window: int) -> np.ndarray:
    if not xs:
        return np.array([])
    arr = np.asarray(xs, dtype=np.float64)
    out = np.empty_like(arr)
    cum = np.cumsum(np.insert(arr, 0, 0.0))
    for i in range(len(arr)):
        j = max(0, i - window + 1)
        out[i] = (cum[i + 1] - cum[j]) / (i - j + 1)
    return out

def reset_env(env):
    out = env.reset()
    if isinstance(out, tuple):
        return out[0]

```

```

    return out

def step_env(env, action):
    out = env.step(action)
    if len(out) == 5:
        next_state, reward, terminated, truncated, _ = out
        done = bool(terminated or truncated)
    else:
        next_state, reward, done, _ = out
    return next_state, float(reward), done

def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--episodes", type=int, default=3000)
    ap.add_argument("--gamma", type=float, default=0.98)
    ap.add_argument("--lr", type=float, default=0.0002)
    ap.add_argument("--plot", action="store_true")
    args = ap.parse_args()

    env = gym.make("CartPole-v0")
    agent = Agent(gamma=args.gamma, lr=args.lr)
    reward_history: list[float] = []

    for episode in range(args.episodes):
        state = reset_env(env)
        done = False
        total_reward = 0.0

        while not done:
            action, prob = agent.get_action(np.asarray(state,
dtype=np.float32))
            next_state, reward, done = step_env(env, action)
            agent.add(reward, prob)
            state = next_state
            total_reward += reward

            agent.update()
            reward_history.append(total_reward)

            if episode % 100 == 0:
                print(f"episode :{episode}, total reward : {total_reward}")

    if args.plot:
        fig, axes = plt.subplots(1, 2, figsize=(11, 4))
        axes[0].plot(reward_history)
        axes[0].set_xlabel("episode")

```

```

axes[0].set_ylabel("total reward")
axes[0].set_title("Episode - total reward")
axes[0].grid(True, alpha=0.3)

axes[1].plot(moving_average(reward_history, 100))
axes[1].set_xlabel("episode")
axes[1].set_ylabel("total reward")
axes[1].set_title("Episode - total reward (100 회 평균)")
axes[1].grid(True, alpha=0.3)

fig.tight_layout()
plt.show()

env.close()

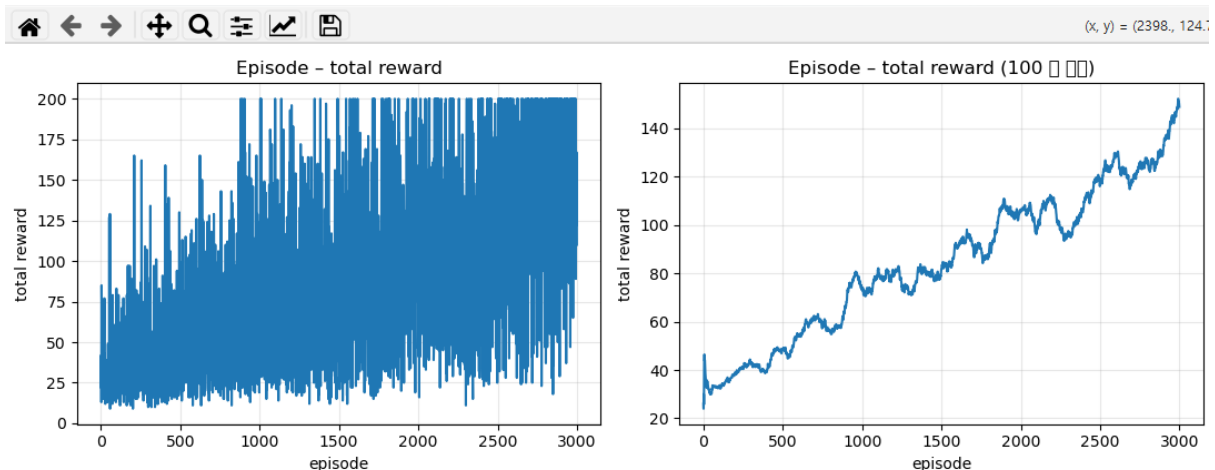
if __name__ == "__main__":
    main()

```

```

logger.deprecation(
episode : 0, total reward : 10.0
episode : 100, total reward : 18.0
episode : 200, total reward : 14.0
episode : 300, total reward : 52.0
episode : 400, total reward : 19.0
episode : 500, total reward : 16.0
episode : 600, total reward : 63.0
episode : 700, total reward : 19.0
episode : 800, total reward : 50.0
episode : 900, total reward : 48.0
episode : 1000, total reward : 54.0
episode : 1100, total reward : 50.0
episode : 1200, total reward : 42.0
episode : 1300, total reward : 163.0
episode : 1400, total reward : 195.0
episode : 1500, total reward : 61.0
episode : 1600, total reward : 51.0
episode : 1700, total reward : 200.0
episode : 1800, total reward : 65.0
episode : 1900, total reward : 130.0
episode : 2000, total reward : 57.0
episode : 2100, total reward : 111.0
episode : 2200, total reward : 55.0
episode : 2300, total reward : 51.0
episode : 2400, total reward : 54.0
episode : 2500, total reward : 101.0
episode : 2600, total reward : 78.0
episode : 2700, total reward : 68.0
episode : 2800, total reward : 33.0
episode : 2900, total reward : 78.0

```



```
"""
```

실습 #3 – Actor-Critic (CartPole, 1-step TD)

PDF: 10-Policy Gradient Method – slides 19~26

```
loss_v = (R_t + γ V_w(S_{t+1}) - V_w(S_t))^2
```

```
loss_pi = -(R_t + γ V_w(S_{t+1}) - V_w(S_t)) · log π_θ(A_t|S_t)
```

실행:

```
python Actor_critic2.py
```

```
python Actor_critic2.py --plot
```

```
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
try:
```

```
    import gym
```

```
except ModuleNotFoundError:
```

```
    import gymnasium as gym
```

```
from dezero import Model
```

```
from dezero import optimizers
```

```
import dezero.functions as F
```

```
import dezero.layers as L
```

```
class PolicyNet(Model):
```

```
    """Actor – policy network  $\pi_\theta(a|s)$ ."""
```

```
    def __init__(self, action_size: int = 2):
```

```
        super().__init__()
```

```
        self.l1 = L.Linear(128)
```

```
        self.l2 = L.Linear(action_size)
```

```
    def forward(self, x):
```

```
        x = F.relu(self.l1(x))
```

```
        x = F.softmax(self.l2(x))
```

```
        return x
```

```
class ValueNet(Model):
```

```
    """Critic – value network  $V_w(s)$ ."""
```

```
    def __init__(self):
```

```
        super().__init__()
```

```

        self.l1 = L.Linear(128)
        self.l2 = L.Linear(1)

    def forward(self, x):
        x = F.relu(self.l1(x))
        x = self.l2(x)
        return x

class Agent:
    def __init__(
        self,
        action_size: int = 2,
        gamma: float = 0.98,
        lr_pi: float = 0.0002,
        lr_v: float = 0.0005,
    ):
        self.gamma = gamma
        self.action_size = action_size

        self.pi = PolicyNet(action_size)
        self.v = ValueNet()
        self.optimizer_pi = optimizers.Adam(lr_pi)
        self.optimizer_v = optimizers.Adam(lr_v)
        self.optimizer_pi.setup(self.pi)
        self.optimizer_v.setup(self.v)

    def get_action(self, state: np.ndarray) -> tuple[int, object]:
        state = state[np.newaxis, :]
        probs = self.pi(state)
        probs = probs[0]
        action = int(np.random.choice(self.action_size, p=probs.data))
        return action, probs[action]

    def update(self, state, action_prob, reward, next_state, done) -> None:
        next_state = next_state[np.newaxis, :]
        target = reward + self.gamma * self.v(next_state) * (1 - done)
        target.unchain()

        v = self.v(state[np.newaxis, :])
        loss_v = F.mean_squared_error(v, target)

        delta = target - v
        delta.unchain()
        loss_pi = -F.log(action_prob) * delta

        self.v.cleargrads()
        self.pi.cleargrads()

```

```

        loss_v.backward()
        loss_pi.backward()
        self.optimizer_v.update()
        self.optimizer_pi.update()

def moving_average(xs: list[float], window: int) -> np.ndarray:
    if not xs:
        return np.array([])
    arr = np.asarray(xs, dtype=np.float64)
    out = np.empty_like(arr)
    cum = np.cumsum(np.insert(arr, 0, 0.0))
    for i in range(len(arr)):
        j = max(0, i - window + 1)
        out[i] = (cum[i + 1] - cum[j]) / (i - j + 1)
    return out

def reset_env(env):
    out = env.reset()
    if isinstance(out, tuple):
        return out[0]
    return out

def step_env(env, action):
    out = env.step(action)
    if len(out) == 5:
        next_state, reward, terminated, truncated, _ = out
        done = bool(terminated or truncated)
        dead = int(terminated)
    else:
        next_state, reward, done, info = out
        truncated = bool(info.get("TimeLimit.truncated", False)) if
isinstance(info, dict) else False
        dead = int(bool(done and not truncated))
    return next_state, float(reward), done, dead

def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--episodes", type=int, default=3000)
    ap.add_argument("--gamma", type=float, default=0.98)
    ap.add_argument("--lr_pi", type=float, default=0.0002)
    ap.add_argument("--lr_v", type=float, default=0.0005)
    ap.add_argument("--plot", action="store_true")
    args = ap.parse_args()

```

```

env = gym.make("CartPole-v0")
agent = Agent(gamma=args.gamma, lr_pi=args.lr_pi, lr_v=args.lr_v)
reward_history: list[float] = []

for episode in range(args.episodes):
    state = reset_env(env)
    done = False
    total_reward = 0.0

    while not done:
        action, prob = agent.get_action(np.asarray(state,
dtype=np.float32))
        next_state, reward, done, dead = step_env(env, action)
        agent.update(state, prob, reward, next_state, dead)
        state = next_state
        total_reward += reward

    reward_history.append(total_reward)

    if episode % 100 == 0:
        print(f"episode :{episode}, total reward : {total_reward}")

if args.plot:
    fig, axes = plt.subplots(1, 2, figsize=(11, 4))
    axes[0].plot(reward_history)
    axes[0].set_xlabel("episode")
    axes[0].set_ylabel("total reward")
    axes[0].set_title("Actor-Critic - Episode - total reward")
    axes[0].grid(True, alpha=0.3)

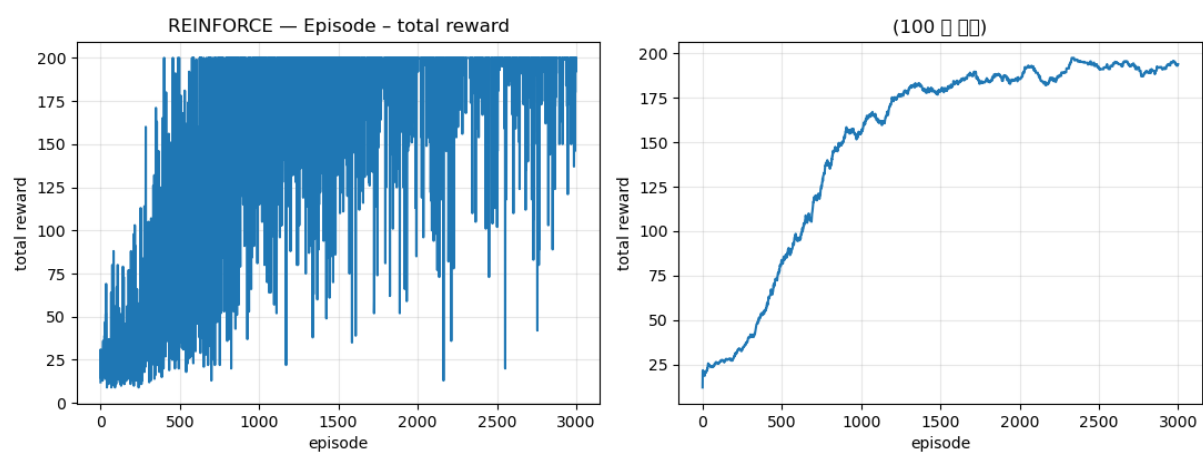
    axes[1].plot(moving_average(reward_history, 100))
    axes[1].set_xlabel("episode")
    axes[1].set_ylabel("total reward")
    axes[1].set_title("(100 회 평균)")
    axes[1].grid(True, alpha=0.3)

    fig.tight_layout()
    plt.show()

env.close()

if __name__ == "__main__":
    main()

```

out of date, you should consider upgrading to version v1.1.

```
logger.deprecation(  
episode :0, total reward : 12.0  
episode :100, total reward : 14.0  
episode :200, total reward : 54.0  
episode :300, total reward : 41.0  
episode :400, total reward : 93.0  
episode :500, total reward : 75.0  
episode :600, total reward : 110.0  
episode :700, total reward : 185.0  
episode :800, total reward : 50.0  
episode :900, total reward : 115.0  
episode :1000, total reward : 200.0  
episode :1100, total reward : 200.0  
episode :1200, total reward : 200.0  
episode :1300, total reward : 199.0  
episode :1400, total reward : 156.0  
episode :1500, total reward : 159.0  
episode :1600, total reward : 200.0  
episode :1700, total reward : 200.0  
episode :1800, total reward : 200.0  
episode :1900, total reward : 200.0  
episode :2000, total reward : 200.0  
episode :2100, total reward : 200.0  
episode :2200, total reward : 200.0  
episode :2300, total reward : 200.0  
episode :2400, total reward : 200.0  
episode :2500, total reward : 200.0  
episode :2600, total reward : 200.0  
episode :2700, total reward : 200.0  
episode :2800, total reward : 200.0  
episode :2900, total reward : 200.0  
episode :3000, total reward : 200.0
```

Add to C

```
"""
```

실습 #3 – Actor-Critic (CartPole, 1-step TD)

PDF: 10-Policy Gradient Method – slides 19~26

```
loss_v = (R_t + γ V_w(S_{t+1}) - V_w(S_t))^2
```

```
loss_pi = -(R_t + γ V_w(S_{t+1}) - V_w(S_t)) · log π_θ(A_t|S_t)
```

실행:

```
python Actor_critic2.py
```

```
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
try:
```

```
    import gym
```

```
except ModuleNotFoundError:
```

```
    import gymnasium as gym
```

```
from dezero import Model
```

```
from dezero import optimizers
```

```
import dezero.functions as F
```

```
import dezero.layers as L
```

```
class PolicyNet(Model):
```

```
    """Actor – policy network  $\pi_\theta(a|s)$ ."""
```

```
    def __init__(self, action_size: int = 2):
```

```
        super().__init__()
```

```
        self.l1 = L.Linear(128)
```

```
        self.l2 = L.Linear(action_size)
```

```
    def forward(self, x):
```

```
        x = F.relu(self.l1(x))
```

```
        x = F.softmax(self.l2(x))
```

```
        return x
```

```
class ValueNet(Model):
```

```
    """Critic – value network  $V_w(s)$ ."""
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.l1 = L.Linear(128)
```

```

        self.l2 = L.Linear(1)

    def forward(self, x):
        x = F.relu(self.l1(x))
        x = self.l2(x)
        return x

class Agent:
    def __init__(
        self,
        action_size: int = 2,
        gamma: float = 0.98,
        lr_pi: float = 0.0002,
        lr_v: float = 0.0005,
    ):
        self.gamma = gamma
        self.action_size = action_size

        self.pi = PolicyNet(action_size)
        self.v = ValueNet()
        self.optimizer_pi = optimizers.Adam(lr_pi)
        self.optimizer_v = optimizers.Adam(lr_v)
        self.optimizer_pi.setup(self.pi)
        self.optimizer_v.setup(self.v)

    def get_action(self, state: np.ndarray) -> tuple[int, object]:
        state = state[np.newaxis, :]
        probs = self.pi(state)
        probs = probs[0]
        action = int(np.random.choice(self.action_size, p=probs.data))
        return action, probs[action]

    def update(self, state, action_prob, reward, next_state, done) -> None:
        next_state = next_state[np.newaxis, :]
        target = reward + self.gamma * self.v(next_state) * (1 - done)
        target.unchain()

        v = self.v(state[np.newaxis, :])
        loss_v = F.mean_squared_error(v, target)

        delta = target - v
        delta.unchain()
        loss_pi = -F.log(action_prob) * delta

        self.v.cleargrads()
        self.pi.cleargrads()
        loss_v.backward()

```

```

        loss_pi.backward()
        self.optimizer_v.update()
        self.optimizer_pi.update()

def moving_average(xs: list[float], window: int) -> np.ndarray:
    if not xs:
        return np.array([])
    arr = np.asarray(xs, dtype=np.float64)
    out = np.empty_like(arr)
    cum = np.cumsum(np.insert(arr, 0, 0.0))
    for i in range(len(arr)):
        j = max(0, i - window + 1)
        out[i] = (cum[i + 1] - cum[j]) / (i - j + 1)
    return out

def reset_env(env):
    out = env.reset()
    if isinstance(out, tuple):
        return out[0]
    return out

def step_env(env, action):
    out = env.step(action)
    if len(out) == 5:
        next_state, reward, terminated, truncated, _ = out
        done = bool(terminated or truncated)
        dead = int(terminated)
    else:
        next_state, reward, done, info = out
        truncated = bool(info.get("Timelimit.truncated", False)) if
isinstance(info, dict) else False
        dead = int(bool(done and not truncated))
    return next_state, float(reward), done, dead

def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--episodes", type=int, default=3000)
    ap.add_argument("--gamma", type=float, default=0.98)
    ap.add_argument("--lr_pi", type=float, default=0.0002)
    ap.add_argument("--lr_v", type=float, default=0.0005)
    args = ap.parse_args()

    env = gym.make("CartPole-v0")
    agent = Agent(gamma=args.gamma, lr_pi=args.lr_pi, lr_v=args.lr_v)

```

```

reward_history: list[float] = []

for episode in range(args.episodes):
    state = reset_env(env)
    done = False
    total_reward = 0.0

    while not done:
        action, prob = agent.get_action(np.asarray(state,
dtype=np.float32))
        next_state, reward, done, dead = step_env(env, action)
        agent.update(state, prob, reward, next_state, dead)
        state = next_state
        total_reward += reward

    reward_history.append(total_reward)

    if episode % 100 == 0:
        print(f"episode :{episode}, total reward : {total_reward}")

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].plot(reward_history)
axes[0].set_xlabel("episode")
axes[0].set_ylabel("total reward")
axes[0].set_title("Actor-Critic - Episode - total reward")
axes[0].grid(True, alpha=0.3)

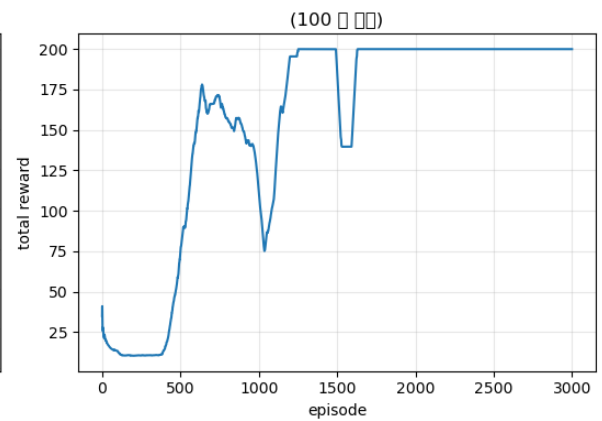
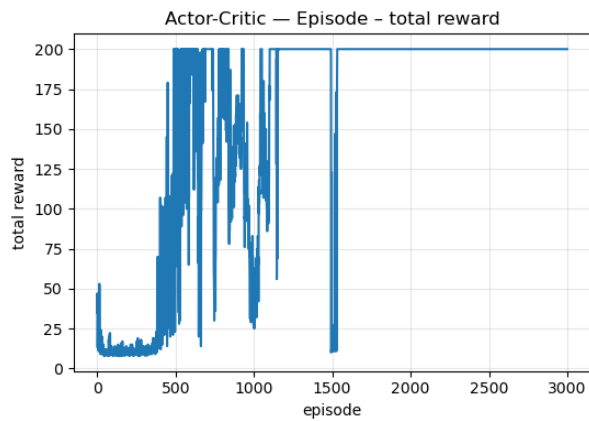
axes[1].plot(moving_average(reward_history, 100))
axes[1].set_xlabel("episode")
axes[1].set_ylabel("total reward")
axes[1].set_title("(100 회 평균)")
axes[1].grid(True, alpha=0.3)

fig.tight_layout()
plt.show()

env.close()

if __name__ == "__main__":
    main()

```



out of date. You should consider upgrading to version v1 .

logger.deprecation(

episode :0, total reward : 35.0

episode :100, total reward : 10.0

episode :200, total reward : 9.0

episode :300, total reward : 9.0

episode :400, total reward : 32.0

episode :500, total reward : 200.0

episode :600, total reward : 197.0

episode :700, total reward : 200.0

episode :800, total reward : 186.0

episode :900, total reward : 171.0

episode :1000, total reward : 51.0

episode :1100, total reward : 172.0

episode :1200, total reward : 200.0

episode :1300, total reward : 200.0

episode :1400, total reward : 200.0

episode :1500, total reward : 24.0

episode :1600, total reward : 200.0

episode :1700, total reward : 200.0

episode :1800, total reward : 200.0

episode :1900, total reward : 200.0

episode :2000, total reward : 200.0

episode :2100, total reward : 200.0

episode :2200, total reward : 200.0

episode :2300, total reward : 200.0

episode :2400, total reward : 200.0

episode :2500, total reward : 200.0

episode :2600, total reward : 200.0

episode :2700, total reward : 200.0

episode :2800, total reward : 200.0

episode :2900, total reward : 200.0